SECP3133 HIGH PERFORMANCE DATA PROCESSING

SEMESTER 2 2024/2025

PROJECT 2

Real-Time Sentiment Analysis on New Straits Times News using Apache Spark and Kafka

SECTION	02
GROUP	GROUP 5
GROUP MEMBERS	1. CAMILY TANG JIA LEI (A22EC0039) 2. NG SHU YU (A22EC0228) 3. WONG QIAO YING (A22EC0118) 4. MUHAMMAD LUQMAN HAKIM (A22EC0086)
LECTURER	DR. ARYATI BINTI BAKRI
SUBMISSION DATE	10 JUNE 2025

Table of Contents

1.0 Introduction.....1

1.1 Background.....2

1.2 Objectives.....3

1.3 Scope.....4

2.0 Data Acquisition and Preprocessing.....6

2.1 Sources.....7

2.2 Tools.....8

2.3 Cleaning Steps.....9

2.3.1 Preprocessing for Spark Logistic Regression (ML Pipeline).....9

2.3.2 Preprocessing for LSTM Deep Learning Model.....10

2.3.3 Sentiment Labelling.....10

3.0 Sentiment Model Development.....12

3.1 Model Choice.....13

3.1.1 Logistic Regression with TF-IDF (Apache Spark MLlib).....13

3.1.2 LSTM Neural Network (Keras with TensorFlow Backend).....14

3.2 Training Process.....15

3.2.1 Data Preparation.....15

3.2.2 Spark Model Training.....15

3.2.3 LSTM Model Training.....16

3.3 Evaluation.....17

4.0 Apache System Architecture.....18

5.0 Analysis and Results.....20

6.0 Optimisation and Comparison.....22

6.1 Model and Architecture Improvements.....23

6.2 Real-Time vs Batch Evaluation.....24

6.3 Prediction Confidence Analysis.....25
6.3.1 LSTM Probability Distribution.....25
6.3.2 Spark Logistic Regression Probability Distribution.....25
6.4 Summary of Comparative Insights.....26
7.0 Conclusion.....27
7.1 Future Work.....28
8.0 References.....29
9.0 Appendices.....30
9.1 Appendix A.....30
9.2 Appendix B.....40
9.3 Appendix C.....41
9.4 Appendix D.....43

1.0 Introduction

With the development of digital times, people are increasingly inclined to express their thoughts, feelings, and opinions on social media, as well as in product comments on e-commerce platforms and news comments. Such user-generated content often encapsulates valuable public opinion that can be exploited to facilitate decisions in many domains, including business analysis, marketing campaigns, political analysis, and crisis management. But the sheer volume, velocity, and variety of this data are overwhelming, creating much noise. In other words, the traditional data analytics paradigm that heavily depends on batch processing of static datasets does not meet global demand for speed and low latency. Such a restriction has sparked the need for real-time sentiment analysis systems that can cope with large amounts of data streams in real time.

This report meets that requirement by implementing a streaming sentiment analysis pipeline on two popular open-source technologies for big data, which are Apache Spark and Apache Kafka. With the help of these tools, the system can consume, stream, and analyse processing data streams nearly in real time. A sentiment classification system based on machine learning and deep learning models is prepared to be able to compare the performance between lightweight and heavier models. Furthermore, natural language processing (NLP) methods are used to preprocess the textual data in order to improve the model's performance. Final outputs are persisted and visualised through popular analytical tools such as Elasticsearch and Kibana, allowing the tracking of the trends in sentiment in real-time.

1.1 Background

Sentiment analysis (SA), or opinion mining, is the task of identifying and understanding the sentiment within text to extract the attitudes, opinions, and emotions expressed in it (Medhat et al. 2014). Typical sentiment analysis techniques, which process a limited number of documents in batch processing, do not possess the capability to handle the continuous, large-scale data streams from services such as Twitter and news outlets. Real-time sentiment analyses can provide immediate insights by monitoring changes in public opinion and facilitating decisions in real time.

Well-known open-source technologies, such as Apache Kafka and Apache Spark, are commonly used to enable real-time data processing. Kafka is known for its high throughput and fault tolerance and is used to stream the live data into Spark, which is a distributed in-memory engine ideal for real-time analytics (e.g., Zaharia et.al, 2016). These technologies, combined with formal statistical modelling methods (e.g., logistic regression (LR) models in traditional machine learning) and deep learning models (e.g, Long Short-Term Memory (LSTM) networks), form an effective pipeline for real-time sentiment classification.

In the Malaysian context, real-time sentiment monitoring allows the understanding of local trends such as product ratings on online sale platforms like Shopee or reactions to any given political or social event on social media. Organisations can make much more informed decisions in advance, better engage with their customers while informing the public, and engage in any new communication strategies and tactics.

1.2 Objectives

This project aims to design and implement a real-time sentiment analysis system focused on analysing public sentiment toward current news topics reported in the Nation category of the *New Straits Times*. The objectives are:

- To create a data pipeline to stream news headlines or article summaries from the *New Straits Times* in real time through web scraping or third-party APIs.

- To clean and preprocess news content using Natural Language Processing (NLP) techniques, including tokenisation, stemming, and stop-word removal.

- To implement and compare two sentiment classification models—Logistic Regression (LR) for fast, interpretable results and Long Short-Term Memory (LSTM) for deep contextual understanding.

- To perform sentiment analysis under both batch and real-time streaming conditions using Apache Spark and Apache Kafka, respectively.

- To store the output sentiment classifications in Elasticsearch and Kibana for results visualisation using interactive dashboards.

- To evaluate model performance across operational modes, highlighting trade-offs in latency, scalability, and accuracy when working with real-time news data.

1.3 Scope

The scope of this project includes the end-to-end development of a sentiment analysis pipeline, which is specifically tailored to real-time news articles from the Nation category of *New Straits Times*. This includes:

- **Data Collection**: Collecting live or regularly updated news headlines or summaries from *New Straits Times*, emphasising Malaysian topics. This will be achieved via web scraping techniques or RSS feed/API integrations.

- **Text Preprocessing**: Using NLP libraries like NLTK, spaCy, or Hugging Face Transformers to clean and standardise text data (e.g., removing punctuation, lowering casing, and stemming).

- **Modelling**: Training and evaluating a traditional ML model (Logistic Regression) and a deep learning model (LSTM) on labelled sentiment data, with a focus on adapting them for news article sentiment.

- **Real-Time Pipeline Integration**: Using Apache Kafka to stream news data and Apache Spark to apply the trained models for real-time inference.

- **Storage and Visualisation**: Indexing sentiment results in Elasticsearch and Kibana as well as building dashboards to track sentiment trends and topic spikes from news articles over time.

- **Performance Evaluation**: Comparing batch vs. streaming model performance using metrics such as accuracy, processing time, and system resource consumption.

This project is limited to textual news data from *the New Straits Times* and does not include user-generated content such as comments. The real-time scope is constrained to headline-level or short-text data, ensuring feasibility within the 4-week timeline and available infrastructure.

2.0 Data Acquisition and Preprocessing

In any data science pipeline, the quality and relevance of the data are important for the effectiveness of the downstream models. The objective of this project is to create a real-time sentiment analysis system based on social or public discourse data, with particular focus on Malaysia. Due to the limitations on publicly accessible and ethical data collection, The New Straits Times is chosen as the primary data source for the study, particularly for its coverage of premium breaking news within Malaysia's political, economic, and social domains.

The data-gathering methodology has been designed to mix historical (batch) and real-time streaming methods to handle different use-case scenarios, such as tracking current events and studying sentiment trends over extended periods. This approach allows for the integration of both past and present sentiment data, which can be valuable in monitoring real-time fluctuations in public opinion. The combination of historical data and live streaming offers the ability to track sentiments and trends effectively over time.

The raw textual data is subject to strict preprocessing rules to ensure an appropriate input to sentiment classification models. The pre-processing process essentially cleans the dataset and normalises the text to some standard, tokenises the data and labels the data for sentiment using rule-based methods. The pre-processing process ends with a processed and labelled dataset that has followed the appropriate pre-processing steps and is input to machine learning or deep learning models.

2.1 Sources

Data for this research study is obtained from the Nation section of The New Straits Times website (https://www.nst.com.my/news/nation). This source is deemed appropriate because it is a reliable and current source of Malaysian news regarding political, economic, and social issues. This study used two retrieval methods:

- **Acquisition in Batch Mode:** A bespoke asynchronous scraper is developed using Playwright to download articles from multiple pages of the website. This approach is able to scrape 20,000 historical articles. Such a large dataset is deemed necessary for the initial training and evaluation of sentiment analysis models, as it provided a comprehensive foundation of past content to allow for meaningful model development and testing.
- **Acquisition in Streaming Mode:** An additional live scraper is created to capture the most recent articles as they are posted. The scraper works by sending requests at a set interval (every 60 seconds), which captures newly released articles as they are published. A comparison mechanism is used to check URLs against a list of previously scraped articles, ensuring that only new articles are processed in real time. This is a useful approach and simulates real data collection, similar to reality, where new information arrives continuously, often at an extremely high volume, requiring the scraping process to work effectively without duplicates as we are processing incoming records and need to distinguish new from previously collected data.

For each collected article (title, description, URL), it is concatenated into a single document string and passed on for additional processing. The preprocessed data is used to feed machine learning models for real-time classification and sentiment analysis.

2.2 Tools

To build a robust data acquisition and preprocessing pipeline, a range of open-source technologies and libraries is integrated. These tools are selected based on their performance, compatibility, and community support, making them suitable for production-grade data pipelines. The following tools and libraries are utilised:

Table 1: Functionality of each tool that is being used

Tool/Library	**Functionality**
Playwright (async)	Automates browser interactions for web scraping of dynamic news content, ensuring that even JavaScript-heavy websites can be scraped effectively.
BeautifulSoup	Parses HTML and extracts relevant tags and text, such as article titles, allowing for fast data extraction from web pages.
Apache Kafka	Delivers real-time message queuing and publishing of recently scraped articles, allowing endless real-time data to flow into the system.
Apache Spark	Processes large-scale text data in both batch and streaming modes, ensuring that the system can handle both historical and real-time data efficiently.
Elasticsearch	Indexes and stores processed data for fast searching and visualisation, enabling the creation of interactive dashboards for sentiment analysis.
NLTK	Performs essential NLP tasks, such as tokenisation, stopword removal, and sentiment scoring, to preprocess text data and prepare it for model input.

All services are containerised in Docker which allowed for modular deployment and ensured that the entire pipeline could be reproducibly executed. Each task, such as the scraping, model training, live prediction, and database storage, had its own container, creating a microservice architecture, which is more scalable and maintainable. This microservice architecture is developed with scaling in mind, providing ease of scalability as the data pipeline increases to ensure large datasets can be effectively processed and managed.

2.3 Cleaning Steps

Text data from web sources often contains noise such as HTML entities, punctuation, and inconsistent formatting. To make the raw text suitable for classification, a comprehensive preprocessing pipeline is applied, which is tailored to the needs of each model architecture. This ensures that the data can be effectively utilised for both machine learning and deep learning models. The complete code for this step can be found in Appendix A (Figure A1).

2.3.1 Preprocessing for Spark Logistic Regression (ML Pipeline)

For the Spark-based logistic regression model, the preprocessing pipeline focuses on converting raw text into a structured numerical representation through TF-IDF (Term Frequency-Inverse Document Frequency). The following preprocessing steps are applied:

- **Removal of Special Characters and Punctuation**: Regular expressions are used to eliminate any unwanted characters, ensuring that only alphanumeric characters and spaces are retained.
- **Lowercasing**: All characters are converted to lowercase to reduce redundancy in word representations, ensuring uniformity in the tokenised text.
- **Tokenization**: NLTK's word_tokenize method is used to split the text into individual words (tokens).
- **Stopword Removal**: Commonly used words (e.g., "the", "is") are removed using the NLTK stopwords corpus. This eliminates words that do not carry significant meaning for sentiment analysis.
- **Stemming**: The Porter Stemmer is applied to reduce words to their root forms (e.g., "running" → "run"). This step reduces dimensionality by combining similar words.
- **Final Output**: The processed and stemmed text is passed into Spark's CountVectorizer and IDF transformer. This approach ensures that the text is represented numerically for input into the logistic regression model.

This preprocessing strategy focuses on dimensionality reduction and model interpretability, which are particularly important for linear models like logistic regression. By eliminating redundancy and irrelevant information, the model can learn more effectively.

2.3.2 Preprocessing for LSTM Deep Learning Model

For the LSTM model, the preprocessing pipeline is designed to retain more contextual richness, given the model's ability to learn sequential dependencies. The following preprocessing steps are applied:

- **Special Character and Punctuation Removal**: Special characters are removed and the text is free of unwanted characters, making it uniform for future processing.
- **Lowercasing**: The text is converted to lowercase while keeping the stopwords to retain the syntactic structure of the text.
- **Tokenization**: The words are tokenized with Keras's Tokenizer functions, which resolved each word to integer index, effectively mapping the text into a numerical form.
- **Padding**: All sequences are padded to a fixed length (e.g., 100 tokens) to ensure a consistent input shape for the LSTM model.

This pre-processing method helps the LSTM model learn subtle linguistic patterns related to sentiment. By preserving stopwords, the model retains the syntactic structure, which is crucial for understanding the context in longer sequences.

2.3.3 Sentiment Labelling

For supervised training, each article must be assigned a ground truth label. The VADER (Valence Aware Dictionary and Sentiment Reasoner) sentiment analyser from NLTK is used to provide a compound polarity score in the range of [-1, 1]. Based on these scores, each article is labelled for the following thresholds:

- **Positive**: If the compound score is greater than or equal to 0.05, the sentiment is labelled positive.
- **Negative**: If the compound score is less than or equal to -0.05, sentiment is labelled negative.
- **Neutral**: All scores in between are labelled neutral.

This rule-based method of labelling enables the autonomous post-hoc labelling of a large quantity of text without manual annotation, thus expediting the model development pipeline. The

post-hoc or autonomous labelling process is very useful in practice since it produces a labelled dataset to train machine learning and deep learning with very little human input. By using VADER, not only does it help to perform sentiment analysis, but it also improves the predictive ability of the model on sentiment.

3.0 Sentiment Model Development

The objective of this task is to train a system capable of classifying news articles as positive, neutral, or negative. Two distinct modelling approaches are followed to perform a comparative analysis between classical machine learning and deep learning methods. The first approach is a logistic regression classifier using TF-IDF representation within the Apache Spark MLlib framework, and the second approach is a deep learning architecture based on Long Short-Term Memory (LSTM) networks with Keras.

This hybrid modelling strategy maximises interpretability and performance. The logistic regression model allows for a transparent and interpretable decision-making process, which is beneficial for debugging and understanding the model decisions. The LSTM model is well suited to manage complex, non-linear patterns in textual data because it can retain the temporal dependencies of word sequences. Therefore, the LSTM is particularly effective at interpreting the complex sentiment of news articles. The training of the scraped dataset can be found in Appendix A (Figure A2).

3.1 Model Choice

To achieve a comprehensive sentiment classification system, two types of models are selected for comparison:

3.1.1 Logistic Regression with TF-IDF (Apache Spark MLlib)

Logistic regression is a linear model that is generally useful for classification because of its simplicity, ability to scale, and interpretable nature. The code in this project utilises logistic regression as part of a Spark Machine Learning pipeline, which consists of the following components:

- **Tokeniser:** The input text is split into individual words.
- **CountVectorizer:** Converts the tokenised words into frequency vectors.
- **IDF (Inverse Document Frequency):** Terms are weighed based on how infrequent they are across the corpus.
- **Logistic Regression Classifier:** The classifier learns the decision boundaries between sentiment classes.

Logistic regression is particularly useful for distributed processing and running large applications on Apache Spark. The TF-IDF transformation allows the model to focus on the important terms of the documents and therefore makes it easier to classify sentiment appropriately.

3.1.2 LSTM Neural Network (Keras with TensorFlow Backend)

The second model is a deep learning classifier consisting of a Long Short-Term Memory (LSTM) architecture. LSTM is a type of recurrent neural network (RNN) that learns from sequences. To capture the nuance of sentiment expressions in news articles, LSTM is chosen since it can learn both long-term dependencies and context. The network architecture consists of the following:

- **Embedding Layer**: Takes each word and maps it to a dense vector space, resulting in a more dynamic representation of the text.

  LSTM Layer: Learns temporal or time relationships between words as it captures the dependencies in the sequences.
- **Dense Output Layer**: Outputs probabilities across the three sentiment classes using softmax activation.

This model was trained using categorical cross-entropy loss and the Adam optimiser. It also performs well with unstructured text data; by learning patterns in the sequences of words, it captures complex dynamics, which is useful for sentiment analysis.

3.2 Training Process

The training process begins once the data, in the form of articles, is collected and preprocessed. Each article is assigned a sentiment label based on the results of VADER sentiment analysis.

3.2.1 Data Preparation

The following is the structure of the preprocessed training data:

- **processed_text_spark**: The text is cleaned and stemmed, ready for use within the Spark pipeline.
- **processed_text_lstm**: The text is cleaned, with stop words preserved, and padded to a fixed length suitable for LSTM.
- **sentiment_numeric**: Numerical labels are assigned to sentiment classes (0 = negative, 1 = neutral, 2 = positive).

The minority classes are upsampled to the size of the majority class in order to solve the issue of class imbalance. By ensuring that the models are trained on a balanced dataset, this enhances their capacity for efficient generalisation.

3.2.2 Spark Model Training

The logistic regression model is trained within a Spark session. The data is split 80:20 (%) for the training and testing portions of the pipeline, respectively. A full pipeline using the training data is formed and the model with the best performance is saved for the live-prediction environment.

Using pipeline staging, which helps ensure modularity and reproducibility, allows for a constant training process. The model will be retrained periodically when new labelled data arrives to maintain its performance over time.

3.2.3 LSTM Model Training

The LSTM model is trained on processed text using Keras. The text is tokenised and padded to a fixed length of max length 100 and split into training and testing datasets (80/20). Early stopping is employed to prevent overfitting, with validation loss monitored throughout the training process.

The network is trained for up to 10 epochs, at a batch size of 64. When the model converged, the trained model and tokeniser are saved for inference during the real-time prediction part of the project.

3.3 Evaluation

To compare the performance and generalisation ability of both the model types on unseen data, the classification model metrics based on logistic regression (via Spark) and the logistic regression model metrics based on the LSTM model (via Keras) are compared. The comparison will be able to quantify how well each model predicts, based on text, the sentiment class (negative, neutral, or positive).

The evaluation is based on the determination of accuracy, precision, recall, and F1-score. The specifications of these criteria allow one to capture several facets of the model performance, which is particularly relevant to multi-class classification data, especially with potential class imbalance data. On the Spark side, MulticlassClassificationEvaluator from Apache is used to compute accuracy. Additionally, classification_report from scikit-learn is applied for further analysis, generating a detailed report on precision, recall, and F1-score across all sentiment classes. Similarly, classification reports for the LSTM model are generated to observe its performance in greater detail.

These metrics are summarised in Appendix B (Table B1), which provides an overview of each metric. The comparison of these models offers a quantitative foundation to evaluate the trade-offs between simplicity, interpretability, and performance in real-time sentiment analysis systems.

4.0 Apache System Architecture

The complete architecture of the real-time sentiment analysis system is shown in Figure 1. It has two primary pipelines: the batch pipeline and the streaming pipeline. The batch pipeline first uses the batch_scraper.py to collect archived news articles from the New Straits Times website and stores those articles in an Elasticsearch index called “news_articles”. Then those articles are used by the batch_trainer.py to train two sentiment models, a logistic regression model using Apache Spark and a long short-term memory (LSTM) model using Keras. During this training process, a tokeniser is also created and saved. This tokeniser is used to prepare text data for the LSTM model.

The streaming pipeline runs perpetually, reading the most recent articles as they are published. “live_scraper.py” scrapes the most recent news headlines and posts them into a Kafka topic “live_news_stream”. The “live_predict.py” script listens to this stream, processes each article, and uses both the pre-trained logistic regression and LSTM models to perform sentiment prediction. It also uses the saved tokeniser to preprocess input text for the LSTM model properly. Prediction results, including sentiment labels and latency metrics, are stored in another Elasticsearch index called “live_predictions”. These results are then visualised using Kibana, giving users a clear and up-to-date view of sentiment trends across incoming news content.

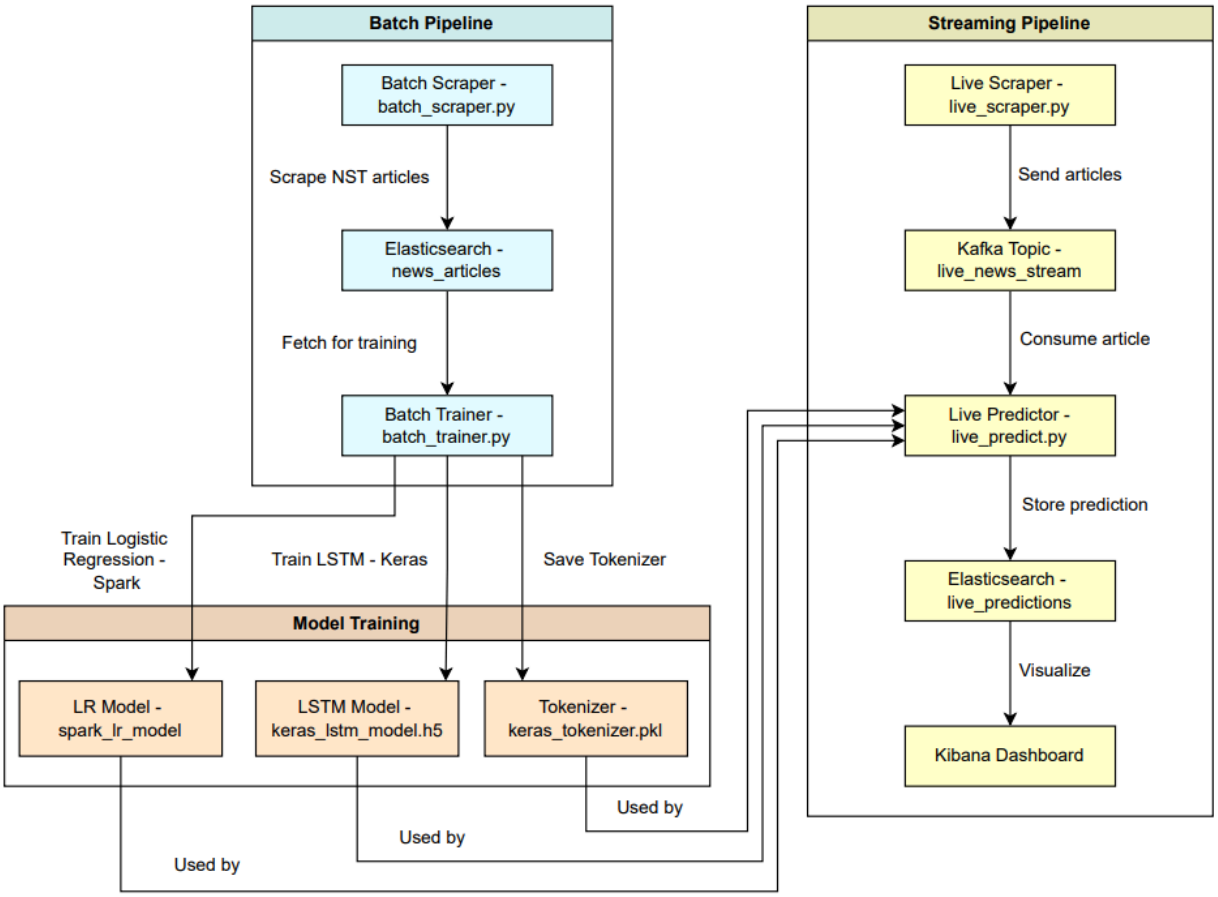

Figure 1. Apache System Architecture Diagram

5.0 Analysis and Results

Figure 2 shows the full Kibana dashboard titled *"New Straits Times Article Real-time Sentiment Analysis Using Logistic Regression Model and LSTM Model"*. This dashboard captures the overall sentiment trends and model comparisons based on 22 streamed news articles. It is designed to help users easily observe how each model interprets article tone and how sentiment shifts over time.

The first chart, a side-by-side horizontal bar graph, compares the frequency with which each model predicted positive, neutral, or negative sentiments. While both models agreed in many cases, the LSTM model leaned more toward positive predictions (13) compared to logistic regression (9). On the other hand, logistic regression showed a stronger tendency to label articles as neutral (7 vs. 2), which may reflect its more conservative classification boundary.

The next two line graphs show how predicted sentiments changed over time, one for each model. These charts break the data into 6-hour intervals and give a visual sense of how sentiment fluctuated as new articles are published. It helps spot bursts of negative news or spikes in positivity during certain parts of the day.

A small summary graph reports that a total of 22 articles are processed during the streaming session. Meanwhile, a doughnut chart breaks down how often the two models agreed or disagreed. The inner ring shows predictions made by logistic regression, while the outer ring reflects what the LSTM model thought for those same articles. This view is particularly useful for quickly spotting inconsistencies, like cases where logistic regression predicted "neutral" but LSTM leaned "positive" or "negative".

Lastly, a sentiment stream table lists each article, its title, timestamp, and the sentiment prediction from both models. It provides a transparent, easy-to-scan record of the system's real-time output. Altogether, the dashboard makes it clear that while both models have their strengths, combining them offers a more complete picture of sentiment trends in the news.

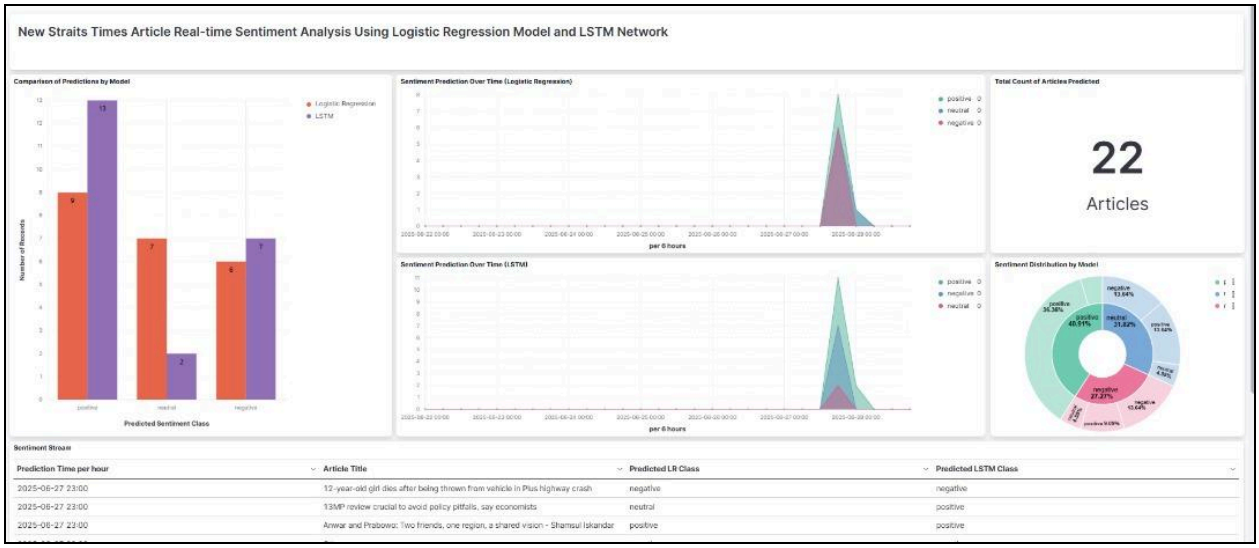

Figure 2. Real-Time Sentiment Analysis Dashboard for New Straits Times Articles Using Logistic Regression and LSTM Models

6.0 Optimisation and Comparison

In this section, the optimisation techniques taken through the model development trajectory and a comparison of the two selected sentiment classification models, namely, Apache Spark Logistic Regression (LR) and a Keras-based Long Short-Term Memory (LSTM) neural network, are described. The comparison is performed in both batch and real-time streaming scenarios by various considerations such as model accuracy, computational efficiency, latency, and prediction confidence.

In this comparative study, the performance as well as the implementation aspects of both approaches are evaluated in different deployment scenarios, particularly in the context of modelling large-scale streaming news data.

6.1 Model and Architecture Improvements

During sentiment classifier development, various critical efforts to optimise the models for robustness and generalisation are made:

- **Text Preprocessing Improvements:** Text cleaning is carried out by our generic pipeline, removing punctuation, converting into lowercase, and then tokenising the input using NLTK. For Spark logistic regression, more stemming and stopword filtering are used to further remove noise and normalise terms.
- **Class Imbalance Management:** The training data indicated an imbalance in terms of the number of positive and negative sentiments and fewer neutral examples. In addition, oversampling the minority class (neutral) is conducted in Spark logistic regression to compensate for this imbalance, and stratified splitting is performed in the LSTM pipeline to maintain the balance of classes.
- **Enhanced Feature Representation (Spark Logistic Regression):** The model pipeline used CountVectorizer and IDF to transform raw text into TF-IDF features. This transformation added semantic weights to features used for training the logistic regression classifier.
- **LSTM (Keras):** Stacked LSTM model by Keras to learn the transitional state across time stamps in the input sequences. We used a 3-layer model including an embedding layer, an LSTM layer, and a dense output layer with softmax activation. The Adam optimiser is used, and the loss function is the sparse categorical crossentropy.
- **Regularisation with Early Stopping (Keras)**: Here, early stopping has been applied to stop the training in case validation loss does not improve further, hence saving on overfitting and computation time.

These improvements resulted in significant advances in classification accuracy and in the ability of the model to generalise to unseen data.

6.2 Real-Time vs Batch Evaluation

Both models are assessed under batch training mode and real-time streaming mode, capturing key metrics such as average latency, prediction counts, and throughput.

Table 2: Metric Evaluation for each model and each mode

Metrics	**Spark Logistic Regression (Batch)**	**Keras LSTM (Batch)**	**Spark Logistic Regression (Streaming)**	**Keras LSTM (Streaming)**
Accuracy (approx.)	~80%	~86%	–	–
Avg Latency (ms)	–	–	1325.546 ms	181.187 ms
Positive Predictions	1734	1803	38	41
Negative Predictions	1337	1589	13	30
Neutral Predictions	446	608	24	4
Throughput (records/sec)	–	–	75	75

In real-time inference, the Keras LSTM model showed a significant improvement over the Spark logistic regression model in latency, with a 181.187 ms response time per prediction vs. 1325.546 ms for Spark logistic regression, respectively, on average across sensors. Although both models have approximately equal prediction count ratios of positive and negative sentiments, the LSTM model is found to be more responsive, which makes it well-adapted for streaming time-constrained applications. It is observed that the throughput remains stable around 75 records/s for both models, demonstrating scalability in production streaming settings. The output of this can be found in Appendix C.

6.3 Prediction Confidence Analysis

Furthermore, in addition to classification, the output probability distributions of the positive predictions are investigated to understand the confidence the model gives to its predictions.

6.3.1 LSTM Probability Distribution

The bimodal mode of probability distribution observed from the Keras LSTM model had a peak close to 0 or 1. This observation suggests that our model made frequent predictions with high confidence, particularly for the class of positive sentiment. Lower-skewed precision ranges are less common, which indicates that there is a confident quality classification approach.

6.3.2 Spark Logistic Regression Probability Distribution

The distribution of the Spark logistic regression model is still more polarised, with a clear peak at extremely low values and a peak at extremely high values. Meanwhile, the Spark model produced more low-confidence predictions compared with LSTM in the above measurements, which could be attributed to the linear nature of this model and the less complex representation of the features.

These confidence profiles indicate that the LSTM is more calibrated for extreme-fidelity decisions, which is useful in domains where false positives or negatives have a significant cost.

6.4 Summary of Comparative Insights

The comparative analysis of Spark Logistic Regression and Keras LSTM in this specific context has shown that each model offers distinct strengths depending on the use case. The Spark LR approach is simple and intuitive, enabling batch processing and integration with distributed data pipelines. Live streaming with the Keras LSTM model has been found to outperform other models in terms of classification performance (both accuracy and real-time).

The latency results indicate that the LSTM model is more efficient, with a faster prediction latency than the Spark logistic regression model. Additionally, the confidence distribution reveals that the LSTM model provides more confident predictions, which is crucial for real-time, time-sensitive, or high-stakes applications.

In summary, these models are diverse and complementary, and their joint use with existing NLP methods meets the robustness and flexibility requirements of the pipeline for both batch and real-world analysis.

7.0 Conclusion

This project has realised and demonstrated an integrated dual-mode sentiment analysis system for batch and real-time news article streams. Utilising Apache Spark for scalable batch processing and Keras LSTM for real-time efficient inferencing, the architecture offers a perspective on a hybrid approach to scaling the system to reach the ideal point between accuracy, latency, and operational flexibility.

Batch is used to perform full training, feature preprocessing, and model scoring with historical news data, and real-time is used to generate near-real-time sentiment predictions on live articles scraped from the web. Its powerful combination with Apache Kafka and Elasticsearch ensured quality messaging and storage and, thus, the availability for smooth data flow and faster indexing that led to downstream analysis or visualisation.

Two different sentiment classification models are compared, which are Spark Logistic Regression and Keras LSTM, each with its pros and cons. The LSTMs performed accuracy- and latency-wise similarly to other models, indicating the suitability of the LSTM for real-time applications. However, the Spark logistic regression model performed well on explanation and scaled batch tasks.

7.1 Future Work

Although the existing framework is well-established, there are several directions we can go in improving performance and usability:

- **Model Enhancements:** Using transformer-based models like BERT could enhance semantic understanding and classification in all sentiment categories.
- **Multilingual:** Expanding the pipeline to multilingual news data, particularly in regional settings, would extend its usefulness.
- **Feedback Loop:** Re-training the models periodically on new data from the live stream with an active learning system could continue to increase adaptability over time.
- **Dashboard integration:** Real-time dashboards showing Sentiment trends could be really helpful for stakeholders in media monitoring, politics, finance, etc.
- **Content Filtering:** Incorporating a stage that screens and eliminates irrelevant or non-news content can enhance the perspective and the prediction accuracy.

Finally, this report shows that batch and streaming sentiment classification can be harmonised into a single, production-ready pipeline. The solution lays a strong foundation for further development towards big data and social and news analytics applications.

8.0 References

Gulli, A., & Pal, S. (2017). *Deep Learning with Keras*. Packt Publishing Ltd.

Kreps, J. (2011). The Log: What every software engineer should know about real-time data's unifying abstraction. *LinkedIn Engineering*. https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying

Medhat, W., Hassan, A., & Korashy, H. (2014). Sentiment analysis algorithms and applications: A survey. *Ain Shams Engineering Journal*, 5(4), 1093–1113. https://doi.org/10.1016/j.asej.2014.04.011

New Straits Times. (n.d.). *Nation*. https://www.nst.com.my/news/nation

Zaharia, M., Das, T., Li, H., Hunter, T., Shenker, S., & Stoica, I. (2016). Discretized streams: Fault-tolerant streaming computation at scale. *Communications of the ACM*, 59(9), 29–35. https://doi.org/10.1145/2934664

9.0 Appendices

9.1 Appendix A

Figures A1 to A4 present the source code implementations for the batch scraping, and cleaning and training of the scraped dataset from the New Straits Times website.

```python
# batch_scraper.py

import asyncio
from playwright.async_api import async_playwright, TimeoutError as PlaywrightTimeoutError
from bs4 import BeautifulSoup
from elasticsearch import Elasticsearch, helpers, NotFoundError

ES_HOST   = 'http://elasticsearch:9200'
ES_INDEX  = 'news_articles'
PAGE_LIMIT = 20000
MAX_BAD_PAGES = 20

async def scrape_articles(limit=PAGE_LIMIT):
    all_articles = []
    total = 0
    page_num = 1
    bad_pages = 0

    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        page = await browser.new_page()

        while total < limit and bad_pages < MAX_BAD_PAGES:
            print(f"[Scraping] Page {page_num} (total scraped: {total}). Bad pages so far: {bad_pages}", flush=True)
            try:
                url = f'https://www.nst.com.my/news/nation?page={page_num}'
                await page.goto(url, wait_until="domcontentloaded")
                await page.wait_for_selector('a.d-flex.article.listing.mb-3.pb-3', timeout=10000)

                soup = BeautifulSoup(await page.content(), 'html.parser')
                articles = soup.find_all(
                    'a',
                    class_='d-flex article listing mb-3 pb-3'
                )

                if not articles:
                    # no articles found ⇒ count as a bad page
                    bad_pages += 1
                    print(f"[Warning] No articles found on page {page_num}.", flush=True)
                else:
                    # successfully scraped at least one article ⇒ reset bad_pages
                    bad_pages = 0
                    for art in articles:
                        if total >= limit:
                            break
                        try:
                            title = art.find('h6', class_='field-title').text.strip()
                            description = art.find(
                                'div', class_='d-block article-teaser'
                            ).text.strip()
                            all_articles.append({
                                'title': title,
                                'description': description
                            })
                            total += 1
                        except Exception:
                            # skip faulty article
                            continue

                page_num += 1
```

```python
            except (PlaywrightTimeoutError, Exception) as e:
                bad_pages += 1
                print(f"[Error] Failed to scrape page {page_num}: {e!r}", flush=True)
                page_num += 1
                # go on to next page

        await browser.close()

    print(f"[Scraping] Finished: scraped {len(all_articles)} articles; stopped after {bad_pages} consecutive bad pages.", flush=True)
    return all_articles

def save_to_elasticsearch(docs):
    es = Elasticsearch(ES_HOST)
    try:
        es.indices.delete(index=ES_INDEX)
        print(f"Deleted old index '{ES_INDEX}'", flush=True)
    except NotFoundError:
        print(f"Index '{ES_INDEX}' did not exist, skipping delete.", flush=True)

    es.indices.create(index=ES_INDEX)
    print(f"Created index '{ES_INDEX}'", flush=True)

    actions = [
        {"_index": ES_INDEX, "_source": doc}
        for doc in docs
    ]
    helpers.bulk(es, actions)
    print(f"Saved {len(docs)} articles to Elasticsearch.", flush=True)

if __name__ == "__main__":
    articles = asyncio.run(scrape_articles())
    save_to_elasticsearch(articles)
    print("Batch scraping and upload complete!")
```

Figure A1: Batch Scraper code

batch_trainer.py > ...

```python
#!/usr/bin/env python3
import os
import re
import pickle

import pandas as pd
import nltk    Import "nltk" could not be resolved
from nltk.corpus import stopwords    Import "nltk.corpus" could not be resolve
from nltk.stem import PorterStemmer    Import "nltk.stem" could not be resolve
from nltk import word_tokenize    Import "nltk" could not be resolved
from nltk.sentiment.vader import SentimentIntensityAnalyzer    Import "nltk.se

from elasticsearch import Elasticsearch

from pyspark.sql import SparkSession
from pyspark.ml.feature import Tokenizer as SparkTokenizer, HashingTF
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline as SparkPipeline
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

from sklearn.model_selection import train_test_split    Import "sklearn.model_
from sklearn.metrics import classification_report    Import "sklearn.metrics"
from sklearn.metrics import classification_report    Import "sklearn.metrics"

from tensorflow.keras.preprocessing.text import Tokenizer as KerasTokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences    Import "t
from tensorflow.keras.models import Sequential    Import "tensorflow.keras.mod
from tensorflow.keras.layers import Embedding, LSTM, Dense    Import "tensorfl
from tensorflow.keras.callbacks import EarlyStopping    Import "tensorflow.ker

# ── Configuration ──────────────────────────────────────────────
ES_HOST          = 'http://elasticsearch:9200'
ES_INDEX         = 'news_articles_batch_uncleaned'
SPARK_MODEL_DIR  = "models/spark_lr_model"
LSTM_MODEL_PATH  = "models/keras_lstm_model.h5"
TOKENIZER_PATH   = "models/keras_tokenizer.pkl"
```

```python
VOCAB_SIZE = 5000
MAXLEN     = 100
EMBED_DIM  = 128
LSTM_UNITS = 128
BATCH_SIZE = 64
EPOCHS     = 10

# ── NLTK Setup ──────────────────────────────────────────────────────
stop_words = set(stopwords.words('english'))
stemmer    = PorterStemmer()
sia        = SentimentIntensityAnalyzer()

# ── Preprocessing Functions ─────────────────────────────────────────
def clean_and_tokenize_for_spark(text: str) -> str:
    """Clean for Spark (stopwords + stemming)."""
    text = re.sub(r'[^A-Za-z0-9\s]', '', text).lower()
    tokens = word_tokenize(text)
    filtered = [w for w in tokens if w not in stop_words]
    stemmed = [stemmer.stem(w) for w in filtered]
    return " ".join(stemmed)

def clean_and_tokenize_for_lstm(text: str) -> str:
    """Clean for LSTM (preserve stopwords)."""
    text = re.sub(r'[^A-Za-z0-9\s]', '', text).lower()
    return text

def sentiment_label(text: str) -> str:
    """Use VADER to assign positive/neutral/negative."""
    score = sia.polarity_scores(text)['compound']
    if score >= 0.05:
        return 'positive'
    elif score <= -0.05:
        return 'negative'
    else:
        return 'neutral'
```

```python
# ── Elasticsearch Fetch ──────────────────────────────────────────────
def fetch_from_es():
    print(f"[Batch] Connecting to ES at {ES_HOST}, index='{ES_INDEX}'", flush=True)
    es = Elasticsearch(ES_HOST)
    resp = es.search(index=ES_INDEX, size=10000, body={"query": {"match_all": {}}}, scroll='2m')
    sid  = resp['_scroll_id']
    hits = resp['hits']['hits']
    all_docs = hits.copy()

    while hits:
        resp = es.scroll(scroll_id=sid, scroll='2m')
        sid  = resp['_scroll_id']
        hits = resp['hits']['hits']
        all_docs.extend(hits)

    print(f"[Batch] Fetched {len(all_docs)} documents from ES.", flush=True)
    return [d['_source'] for d in all_docs]

# ── Spark Model ──────────────────────────────────────────────────────
def train_and_save_spark_model(df: pd.DataFrame):
    from pyspark.sql import SparkSession
    from pyspark.ml.feature import Tokenizer as SparkTokenizer, CountVectorizer, IDF
    from pyspark.ml.classification import LogisticRegression
    from pyspark.ml import Pipeline as SparkPipeline
    from pyspark.ml.evaluation import MulticlassClassificationEvaluator

    print("[Spark] Starting SparkSession...", flush=True)
    spark = SparkSession.builder \
        .appName("NewsMLPipeline") \
        .config("spark.jars.packages", "org.elasticsearch:elasticsearch-spark-30_2.12:7.17.21") \
        .getOrCreate()
```

```python
# Optional: Balance the dataset
print("[Spark] Rebalancing training data (upsampling minority classes)...", flush=True)
majority_df = df[df["sentiment_numeric"] == 2]
minority_df = df[df["sentiment_numeric"] != 2]
resampled_df = pd.concat([
    majority_df,
    minority_df.sample(n=len(majority_df), replace=True, random_state=42)
])

# Create Spark DataFrame from resampled data
sdf = spark.createDataFrame(resampled_df[["processed_text_spark", "sentiment_numeric"]]) \
            .withColumnRenamed("processed_text_spark", "processed_text")

# Define pipeline: Tokenizer → CountVectorizer → IDF → LogisticRegression
tokenizer = SparkTokenizer(inputCol="processed_text", outputCol="words")
cv        = CountVectorizer(inputCol="words", outputCol="rawFeatures", vocabSize=10000)
idf       = IDF(inputCol="rawFeatures", outputCol="features")
lr        = LogisticRegression(featuresCol="features", labelCol="sentiment_numeric", maxIter=50)

pipeline = SparkPipeline(stages=[tokenizer, cv, idf, lr])

# Train-test split and model training
print("[Spark] Training Logistic Regression with TF-IDF...", flush=True)
train, test = sdf.randomSplit([0.8, 0.2], seed=42)
model = pipeline.fit(train)

# Evaluation
print("[Spark] Evaluating model...", flush=True)
preds = model.transform(test)
evaluator = MulticlassClassificationEvaluator(
    labelCol="sentiment_numeric", predictionCol="prediction", metricName="accuracy"
)
acc = evaluator.evaluate(preds)
print(f"[Spark] Test Accuracy = {acc:.4f}", flush=True)
```

```python
    # Full classification report using sklearn
    y_true = preds.select("sentiment_numeric").rdd.flatMap(lambda x: x).collect()
    y_pred = preds.select("prediction").rdd.flatMap(lambda x: x).collect()
    print("[Spark] Classification Report:")
    print(classification_report(y_true, y_pred, target_names=["negative", "neutral", "positive"]))

    # Save the model
    os.makedirs(os.path.dirname(SPARK_MODEL_DIR), exist_ok=True)
    model.write().overwrite().save(SPARK_MODEL_DIR)
    print(f"[Spark] Model saved to {SPARK_MODEL_DIR}", flush=True)

    spark.stop()

# ── Keras LSTM Model ──────────────────────────────────────────────────────
def train_and_save_keras_lstm(df: pd.DataFrame):
    texts  = df["processed_text_lstm"].tolist()
    labels = df["sentiment_numeric"].values

    kt = KerasTokenizer(num_words=VOCAB_SIZE)
    kt.fit_on_texts(texts)
    seqs = kt.texts_to_sequences(texts)
    X    = pad_sequences(seqs, maxlen=MAXLEN)
    y    = labels

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

    print("[Keras] Training 3-class LSTM...", flush=True)
    model = Sequential([
        Embedding(input_dim=VOCAB_SIZE, output_dim=EMBED_DIM, input_length=MAXLEN),
        LSTM(LSTM_UNITS, dropout=0.2, recurrent_dropout=0.2),
        Dense(3, activation='softmax')
    ])
    model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```python
    es_cb = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
    model.fit(X_train, y_train, validation_data=(X_test, y_test),
              epochs=EPOCHS, batch_size=BATCH_SIZE, callbacks=[es_cb], verbose=2)

    # Evaluate and print classification report
    y_pred = model.predict(X_test).argmax(axis=1)
    print("[Keras] Classification Report:")
    print(classification_report(y_test, y_pred, target_names=['negative', 'neutral', 'positive']))

    os.makedirs(os.path.dirname(LSTM_MODEL_PATH), exist_ok=True)
    model.save(LSTM_MODEL_PATH)
    with open(TOKENIZER_PATH, "wb") as f:
        pickle.dump(kt, f)
    print(f"[Keras] Model saved to {LSTM_MODEL_PATH}", flush=True)
    print(f"[Keras] Tokenizer saved to {TOKENIZER_PATH}", flush=True)

# ── Main ──────────────────────────────────────────────────────────────────
def main():
    docs = fetch_from_es()
    if not docs:
        print("[Batch] No documents found. Exiting.", flush=True)
        return

    df = pd.DataFrame(docs)

    # Preprocess + label
    df["combined_text"] = df["title"].fillna("") + " " + df["description"].fillna("")
    df["processed_text_spark"] = df["combined_text"].apply(clean_and_tokenize_for_spark)
    df["processed_text_lstm"]  = df["combined_text"].apply(clean_and_tokenize_for_lstm)
    df["sentiment_label"]      = df["processed_text_lstm"].apply(sentiment_label)
    df["sentiment_numeric"]    = df["sentiment_label"].map({'negative': 0, 'neutral': 1, 'positive': 2})

    print("[Batch] Label distribution:")
    print(df["sentiment_label"].value_counts())
```

```
    # Train models
    train_and_save_spark_model(df)
    train_and_save_keras_lstm(df)

if __name__ == "__main__":
    main()
```

Figure A2: Batch Trainer code

```python
import os
import json
import time
import joblib
import nltk    Import "nltk" could not be resolved
import numpy as np
import psutil # New import for resource usage    Import "psutil" could not be resolved from source
import sys    # New import for sys.stderr

from elasticsearch import Elasticsearch
from kafka import KafkaConsumer    Import "kafka" could not be resolved
from tensorflow.keras.models import load_model    Import "tensorflow.keras.models" could not be resolved
from tensorflow.keras.preprocessing.sequence import pad_sequences    Import "tensorflow.keras.preprocessin
from datetime import datetime, timezone
from pyspark.sql import SparkSession
from pyspark.ml import PipelineModel

# --- Configuration for Metrics ---
THROUGHPUT_REPORT_INTERVAL_SECONDS = 30 # Report throughput every N seconds
MEMORY_REPORT_INTERVAL_RECORDS = 100   # Report memory every N records

print("[Startup] Downloading NLTK resources...")
# (downloads only the first time; quiet to reduce console noise)
# nltk.download('stopwords', quiet=True)
# nltk.download('punkt', quiet=True)
# nltk.download('vader_lexicon', quiet=True) # VADER for batch_trainer, but useful for general NLTK setup
stop_words = set(nltk.corpus.stopwords.words('english'))

def clean_and_tokenize(text):
    import re
    text = re.sub(r'[^A-Za-z0-9\s]', '', text)
    text = text.lower()
    tokens = nltk.word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    return " ".join(tokens)

inv_label_map = {2: 'positive', 1: 'neutral', 0: 'negative'}
```

```python
print("[Startup] Loading models and tokenizer...")
try:
    # Load Spark model (whole pipeline)
    # Get or create SparkSession (important for PySpark inside Docker)
    spark = SparkSession.builder.appName("predictor").getOrCreate()
    spark_model = PipelineModel.load('models/spark_lr_model')
    print("   - Spark Logistic Regression Pipeline loaded.")

    # Load Keras LSTM model and tokenizer
    with open('models/keras_tokenizer.pkl', 'rb') as f:
        tokenizer = joblib.load(f)
    print("   - Keras Tokenizer loaded.")
    lstm_model = load_model('models/keras_lstm_model.h5')
    print("   - Keras LSTM model loaded.")
except Exception as e:
    print(f"[Startup][Error] Failed to load models: {e}", file=sys.stderr)
    sys.exit(1) # Use sys.exit(1) for cleaner exit on startup error
print("[Startup] All models loaded successfully.")

KAFKA_BOOTSTRAP = os.getenv('KAFKA_BOOTSTRAP', 'kafka:9092')
KAFKA_TOPIC = os.getenv('KAFKA_STREAM_TOPIC', 'live_news_stream')

print(f"[Startup] Connecting to Elasticsearch at http://elasticsearch:9200 ...", flush=True)
try:
    es = Elasticsearch('http://elasticsearch:9200')
    if not es.ping():
        raise Exception("Elasticsearch is not reachable!")
    print("[Startup] Connected to Elasticsearch successfully.", flush=True)
except Exception as e:
    print(f"[Startup][Error] Cannot connect to Elasticsearch: {e}", file=sys.stderr)
    sys.exit(1)

print(f"[Startup] Connecting to Kafka at {KAFKA_BOOTSTRAP} (topic: '{KAFKA_TOPIC}') ...", flush=True)
try:
```

```python
    consumer = KafkaConsumer(
        KAFKA_TOPIC,
        bootstrap_servers=KAFKA_BOOTSTRAP,
        value_deserializer=lambda x: json.loads(x.decode('utf-8')),
        auto_offset_reset='latest',
        enable_auto_commit=True,
        group_id="news_predict_group"
    )
    print("[Startup] Kafka consumer created. Waiting for messages ...", flush=True)
except Exception as e:
    print(f"[Startup][Error] Cannot connect to Kafka: {e}", file=sys.stderr)
    sys.exit(1)

# --- Performance Metrics Setup ---
total_records_processed = 0
overall_start_time = time.perf_counter()
last_throughput_report_time = overall_start_time

process = psutil.Process(os.getpid()) # Get current process for memory usage

for msg in consumer:
    article = msg.value
    title = article.get('title', '')
    description = article.get('description', '')
    link = article.get('link', '')
    text = f"{title} {description}".strip()
    print(f"\n[Kafka] Received new article: {title} ({link})")
    processed = clean_and_tokenize(text)
    print("   [Predict] Text cleaned and tokenized.")

    # --- Logistic Regression using PySpark Pipeline ---
    lr_start_time = time.perf_counter()
    test_df = spark.createDataFrame([{"processed_text": processed}])
    predictions_lr_spark = spark_model.transform(test_df)
```

```python
    # Collect prediction and probability
    lr_result = predictions_lr_spark.select("prediction", "probability").first()
    pred_lr = int(lr_result["prediction"])
    pred_lr_prob_vector = lr_result["probability"] # This is a Spark ML Vector
    # Convert Spark ML Vector to Python list for easier logging
    # pred_lr_probs will be a list of floats (e.g., [0.1, 0.2, 0.7] for 3 classes)
    pred_lr_probs = list(pred_lr_prob_vector) if hasattr(pred_lr_prob_vector, 'toArray') else pred_lr_prob_vector.tolist()
    lr_end_time = time.perf_counter()
    lr_latency_ms = (lr_end_time - lr_start_time) * 1000
    # FIX IS HERE: Removed :.4f as pred_lr_probs is a list
    print(f"   [Predict][Spark LR] Prediction: {inv_label_map.get(pred_lr, pred_lr)} (Probabilities: {pred_lr_probs})")
    print(f"   [Predict][Spark LR] Latency: {lr_latency_ms:.2f} ms")


    # --- LSTM ---
    lstm_start_time = time.perf_counter()
    seq = tokenizer.texts_to_sequences([processed])
    pad = pad_sequences(seq, maxlen=100)
    lstm_probabilities = lstm_model.predict(pad, verbose=0)[0]
    pred_lstm = int(np.argmax(lstm_probabilities))
    (variable) lstm_latency_ms: float
    lstm_latency_ms = (lstm_end_time - lstm_start_time) * 1000
    print(f"   [Predict][LSTM] Prediction: {inv_label_map.get(pred_lstm, pred_lstm)} (Probabilities: {lstm_probabilities})")
    print(f"   [Predict][LSTM] Latency: {lstm_latency_ms:.2f} ms")


    total_records_processed += 1

```

```python
    result = {
        'title': title,
        'description': description,
        'link': link,
        'text': text,
        'processed': processed,
        'logreg_pred': pred_lr,
        'logreg_pred_text': inv_label_map.get(pred_lr, 'unknown'),
        'logreg_probabilities': pred_lr_probs, # Store probabilities
        'logreg_latency_ms': lr_latency_ms,     # Store latency
        'lstm_pred': pred_lstm,
        'lstm_pred_text': inv_label_map.get(pred_lstm, 'unknown'),
        'lstm_probabilities': lstm_probabilities.tolist(), # Store probabilities as list
        'lstm_latency_ms': lstm_latency_ms,       # Store latency
        '@timestamp': datetime.now(timezone.utc).isoformat()
    }

    try:
        es.index(index='live_predictions', body=result)
        print("   [Elasticsearch] Prediction result saved successfully.\n")
    except Exception as e:
        print(f"   [Elasticsearch][Error] Failed to save prediction: {e}\n", file=sys.stderr)

    # --- Throughput Reporting ---
    current_time = time.perf_counter()
    elapsed_overall_time = current_time - overall_start_time

    if (current_time - last_throughput_report_time) >= THROUGHPUT_REPORT_INTERVAL_SECONDS:
        overall_throughput = total_records_processed / elapsed_overall_time
        print(f"\n--- Performance Report ({datetime.now().strftime('%H:%M:%S')}) ---", flush=True)
        print(f"  Total Records Processed: {total_records_processed}", flush=True)
        print(f"  Overall Throughput: {overall_throughput:.2f} records/sec", flush=True)
```

```python
        # --- Resource Usage ---
        # Note: This measures the Python process's memory. For full Docker container CPU/RAM, use `docker stats`.
        mem_info = process.memory_info()
        rss_mb = mem_info.rss / (1024 * 1024) # Resident Set Size in MB
        vms_mb = mem_info.vms / (1024 * 1024) # Virtual Memory Size in MB
        print(f"  Current Python Process Memory (RSS): {rss_mb:.2f} MB", flush=True)
        print(f"  Current Python Process Memory (VMS): {vms_mb:.2f} MB", flush=True)

        print("----------------------------------------\n", flush=True)
        last_throughput_report_time = current_time

print("[Shutdown] Exiting streamPredict.")
```

Figure A3: Live Predict code

```python
import asyncio
import json
from kafka import KafkaProducer
from playwright.async_api import async_playwright
from bs4 import BeautifulSoup

KAFKA_BOOTSTRAP = 'kafka:9092'
KAFKA_TOPIC = 'live_news_stream'

async def scrape_latest_articles(page):
    url = 'https://www.nst.com.my/news/nation'
    print(f"[Scraper] Navigating to {url}")
    await page.goto(url, wait_until="domcontentloaded")
    print("[Scraper] Page loaded, waiting for articles selector...")
    await page.wait_for_selector('a.d-flex.article.listing.mb-3.pb-3')
    print("[Scraper] Articles selector found, getting HTML content.")
    content = await page.content()
    print(f"[Scraper] Length of HTML content: {len(content)}")
    soup = BeautifulSoup(content, 'html.parser')
    articles = soup.find_all('a', class_=['d-flex', 'article', 'listing', 'mb-3', 'pb-3'])
    print(f"[Scraper] Found {len(articles)} article elements in HTML.")
    article_details = []
    for idx, article in enumerate(articles):
        try:
            link = article['href']
            if link and link.startswith('/'):
                link = "https://www.nst.com.my" + link
            title = article.find('h6', class_='field-title').text.strip()
            description = article.find('div', class_='d-block article-teaser').text.strip()
            print(f"[Scraper] Article {idx+1}: {title} ({link})")
            article_details.append({
                'title': title,
                'description': description,
                'link': link
            })
        except Exception as e:
            print(f"[Scraper] Skipped an article due to error: {e}")
            continue
    print(f"[Scraper] Returning {len(article_details)} articles from this scrape.")
    return article_details


async def stream_scrape():
    producer = None
    browser = None
    try:
        print("[Kafka] Connecting to Kafka producer...")
        producer = KafkaProducer(
            bootstrap_servers=KAFKA_BOOTSTRAP,
            value_serializer=lambda x: json.dumps(x).encode('utf-8')
        )
        seen_links = set()
        print("[Playwright] Launching browser...")
        async with async_playwright() as p:
            browser = await p.chromium.launch(headless=True)
            page = await browser.new_page()
            while True:
                print("\n[Main Loop] Checking for new articles...")
```

```python
                try:
                    articles = await scrape_latest_articles(page)
                except Exception as e:
                    print(f"[Main Loop] Error during scraping: {e}")
                    await asyncio.sleep(60)
                    continue
                new_articles = [a for a in articles if a['link'] not in seen_links]
                print(f"[Main Loop] Found {len(new_articles)} new articles this round.")
                for article in new_articles:
                    try:
                        producer.send(KAFKA_TOPIC, value=article)
                        print(f"[Kafka] Sent: {article['title']} ({article['link']})", flush=True)
                        seen_links.add(article['link'])
                    except Exception as e:
                        print(f"[Kafka] Failed to send article to Kafka: {e}")
                producer.flush()
                print(f"[Main Loop] Sleeping for 60 seconds before next check.")
                await asyncio.sleep(60)  # Async sleep!
    finally:
        print("[Shutdown] Closing browser and Kafka producer.")
        if browser:
            await browser.close()
        if producer:
            producer.close()

if __name__ == '__main__':
    print("[Startup] Starting the news scraper.")
    asyncio.run(stream_scrape())
```

Figure A4: Live Scraper code

9.2 Appendix B

Table B1 below lists the description and purpose of each metric that is being used during model evaluation.

Table B1: Overview of each metric during model evaluation

Metric	**Description**	**Purpose in Evaluation**
Accuracy	Proportion of correct predictions out of total predictions.	Measures the overall correctness of the model.
Precision	Proportion of correct positive predictions for each class.	Evaluates the model's ability to avoid false positives (i.e., predicting the wrong sentiment).
Recall	Proportion of actual class instances correctly identified.	Evaluates the model's ability to detect all relevant cases (i.e., sentiment coverage).
F1-score	Harmonic mean of precision and recall.	Balances precision and recall; useful when dealing with class imbalance.
Support	Number of actual occurrences of each class in the test set.	Indicates the sample size used to calculate the above metrics for each class.

9.3 Appendix C

Figures C1 to C4 present the output for both batch and streaming modes for models Logistic Regression (LR) and Long Short-Term Memory (LSTM).

```
[Spark] Test Accuracy = 0.8049
[Spark] Classification Report:
              precision    recall  f1-score   support

    negative       0.83      0.85      0.84      1337
     neutral       0.56      0.73      0.64       446
    positive       0.87      0.79      0.83      1734

    accuracy                           0.80      3517
   macro avg       0.76      0.79      0.77      3517
weighted avg       0.82      0.80      0.81      3517

[Keras] Classification Report:
              precision    recall  f1-score   support

    negative       0.89      0.86      0.88      1589
     neutral       0.75      0.73      0.74       608
    positive       0.87      0.90      0.89      1803

    accuracy                           0.86      4000
   macro avg       0.84      0.83      0.83      4000
weighted avg       0.86      0.86      0.86      4000
```

Figure C1: Output for Batch mode for both models

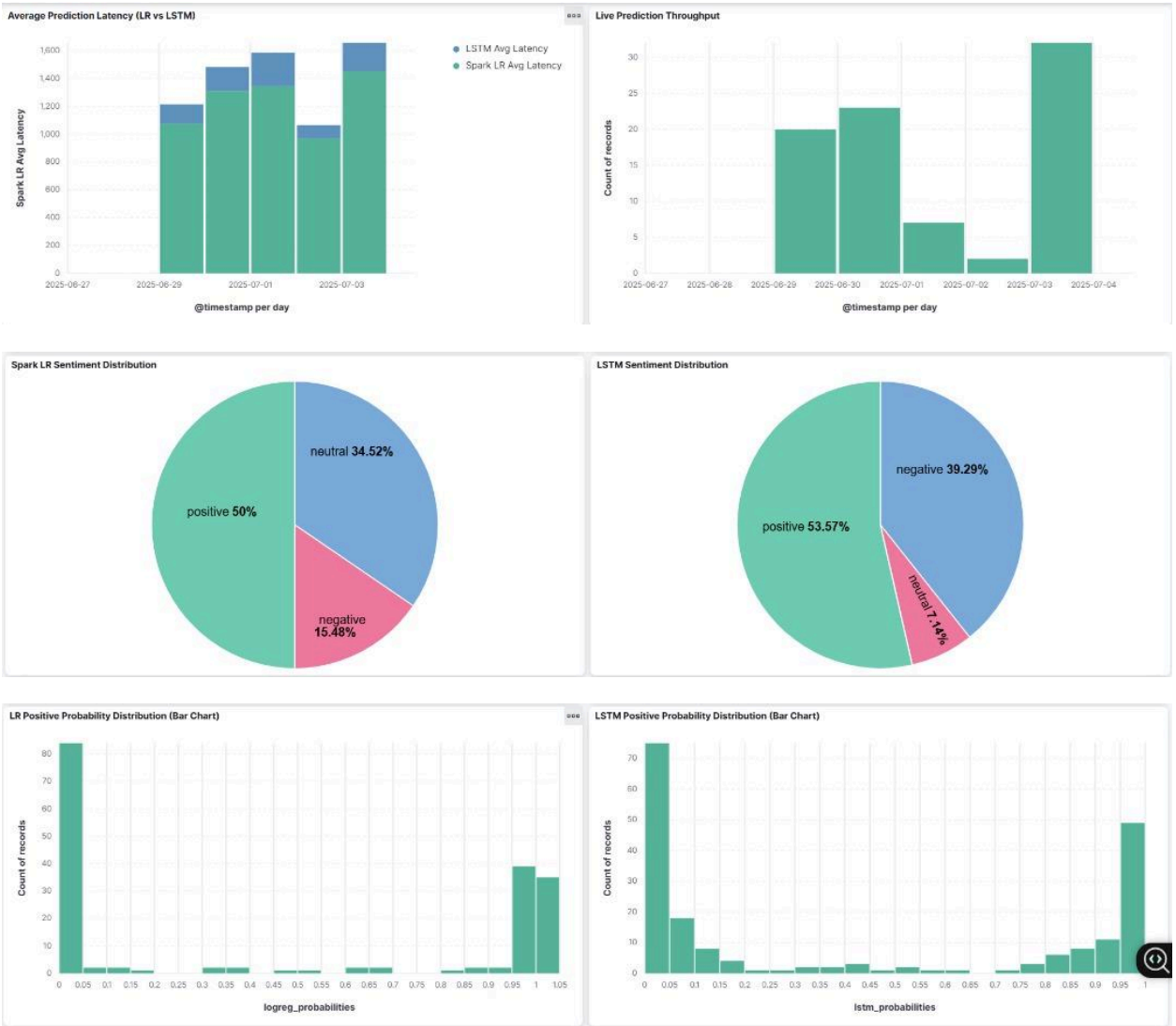

Figure C2: Output for Stream mode for both models

9.4 Appendix D

The full project code and dataset are available at the following GitHub repository: https://github.com/Jingyong14/HPDP02/tree/main/2425/project/p2/Group_5

The full LogBook activity is available at the following Google Sheets link: https://docs.google.com/spreadsheets/d/1yUCRjXETjp0RcAfXREapw924eojlzB8SJwdtpw-X53Q/edit?usp=sharing